

Real-time streaming analytics and machine learning with Apache Flink

UL FRI

Data Science Workshop

Stefan Popov, Domen Požrl

Wednesday, 22 October 2025

Stream Processing with Apache Flink

Fundamentals, Implementation, and
Operation of Streaming
Applications

O'REILLY®

Stream Processing with Apache Flink

Fundamentals, Implementation, and Operation
of Streaming Applications



Fabian Hueske &
Vasiliki Kalavri

Amazon: <https://www.amazon.com/Stream-Processing-Apache-Flink-Implementation/dp/149197429X>

Agenda

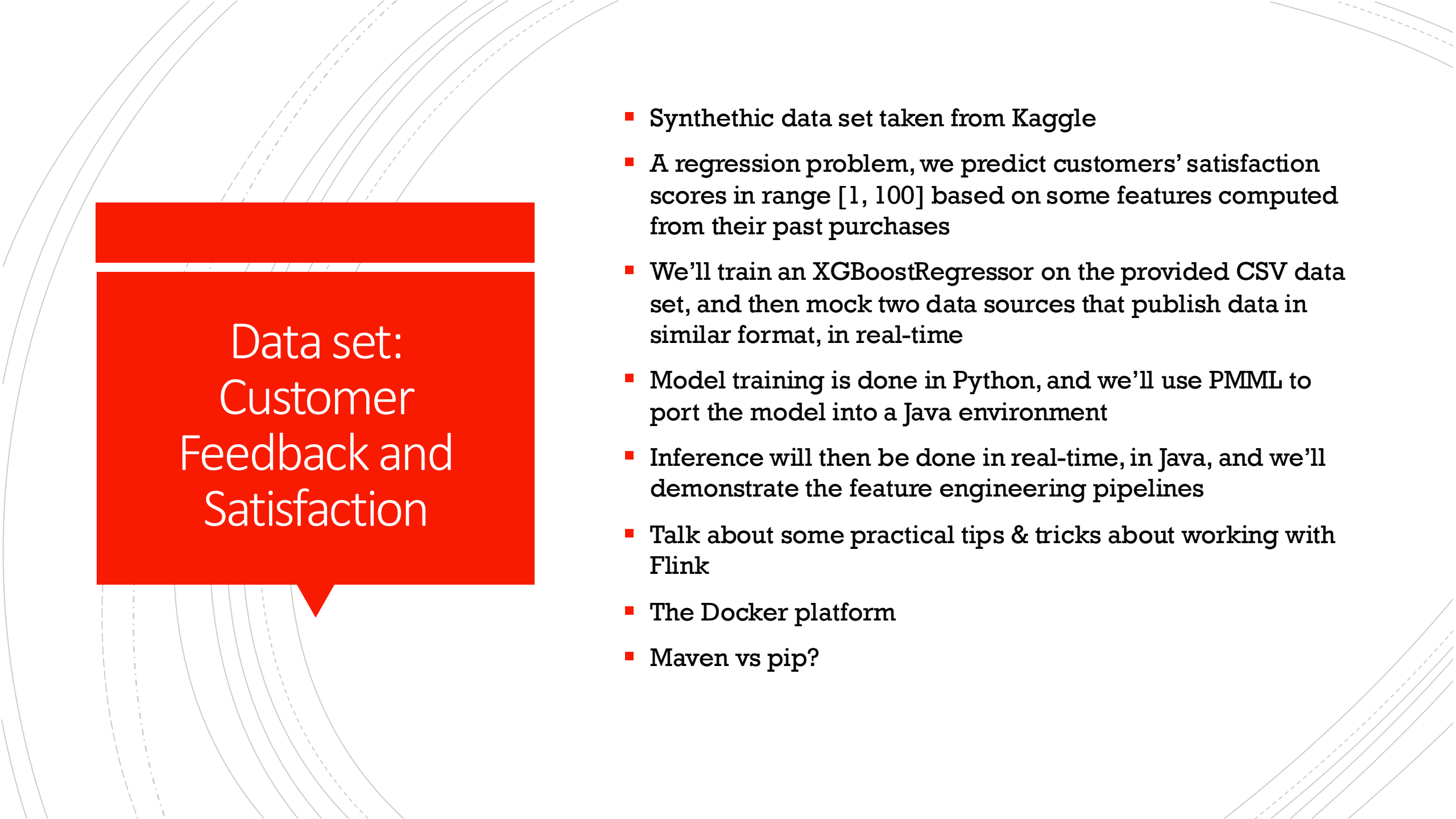
- Workshop/problem introduction
- Flink architecture & memory state
 - Overview of DataStream API
- Machine learning in Flink
- State persistence, fault tolerance
- Building and running a Docker image
- Hands-on exercises

The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. A large red speech bubble is centered on the page, containing the main text.

Workshop/problem introduction

Data set overview, problem setup

Data science with Java

The background of the slide features a series of thin, light gray curved lines that sweep across the frame, creating a sense of motion and depth. These lines are more densely packed on the left side and become sparser towards the right.

Data set: Customer Feedback and Satisfaction

- Synthetic data set taken from Kaggle
- A regression problem, we predict customers' satisfaction scores in range [1, 100] based on some features computed from their past purchases
- We'll train an XGBoostRegressor on the provided CSV data set, and then mock two data sources that publish data in similar format, in real-time
- Model training is done in Python, and we'll use PMML to port the model into a Java environment
- Inference will then be done in real-time, in Java, and we'll demonstrate the feature engineering pipelines
- Talk about some practical tips & tricks about working with Flink
- The Docker platform
- Maven vs pip?

Let's first see
the Python part

Jupyter Notebooks



Different use-cases:
real-time streaming
vs batch
environment

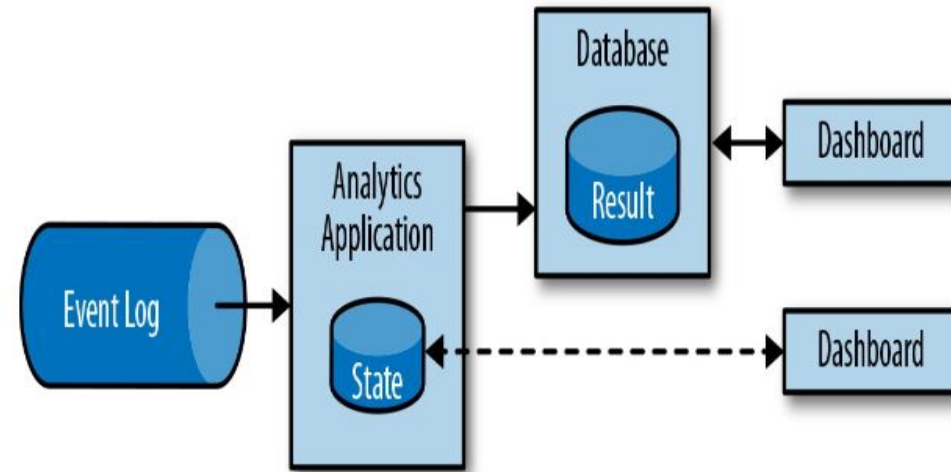


Figure 1-6. A streaming analytics application

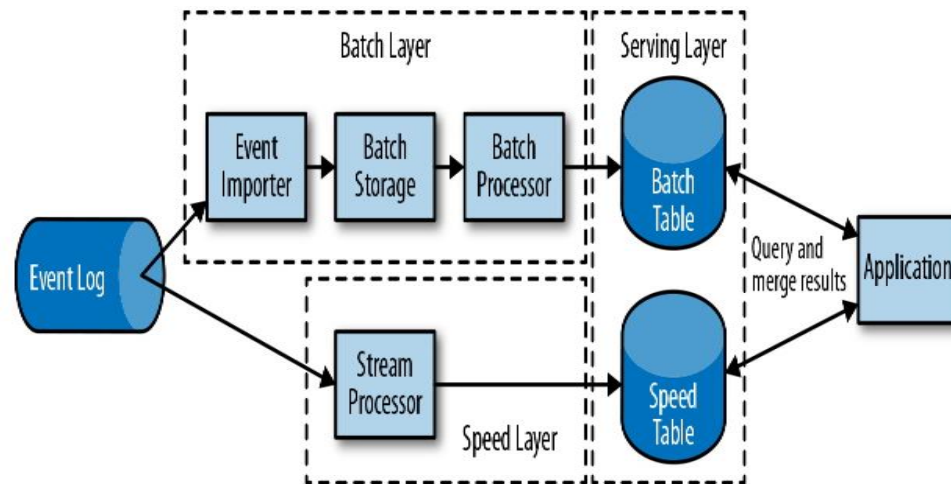
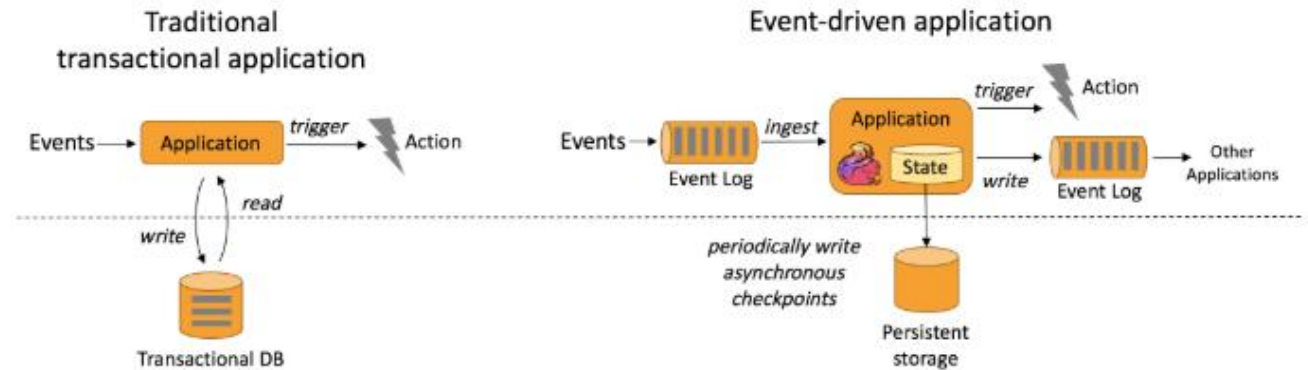
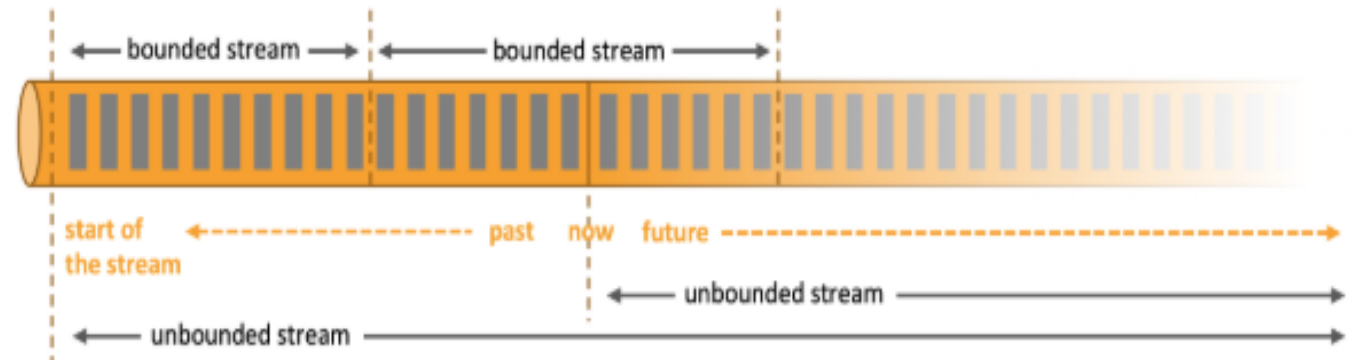


Figure 1-7. The lambda architecture



About Apache Flink



Apache Flink is a framework and distributed processing engine for stateful computations over unbounded and bounded data streams. Flink has been designed to run in all common cluster environments, perform computations at in-memory speed and at any scale.

Flink architecture, overview of DataStream API

Job manager, task manager, execution environment,
data flow graph, Flink UI

Functions: map, flatMap, filter, process, coProcess,
etc.

Logical overview

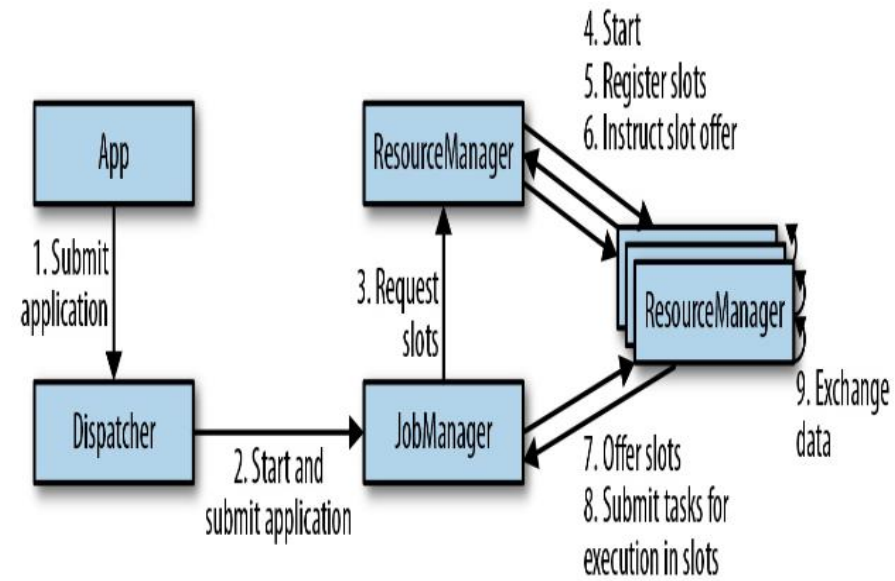
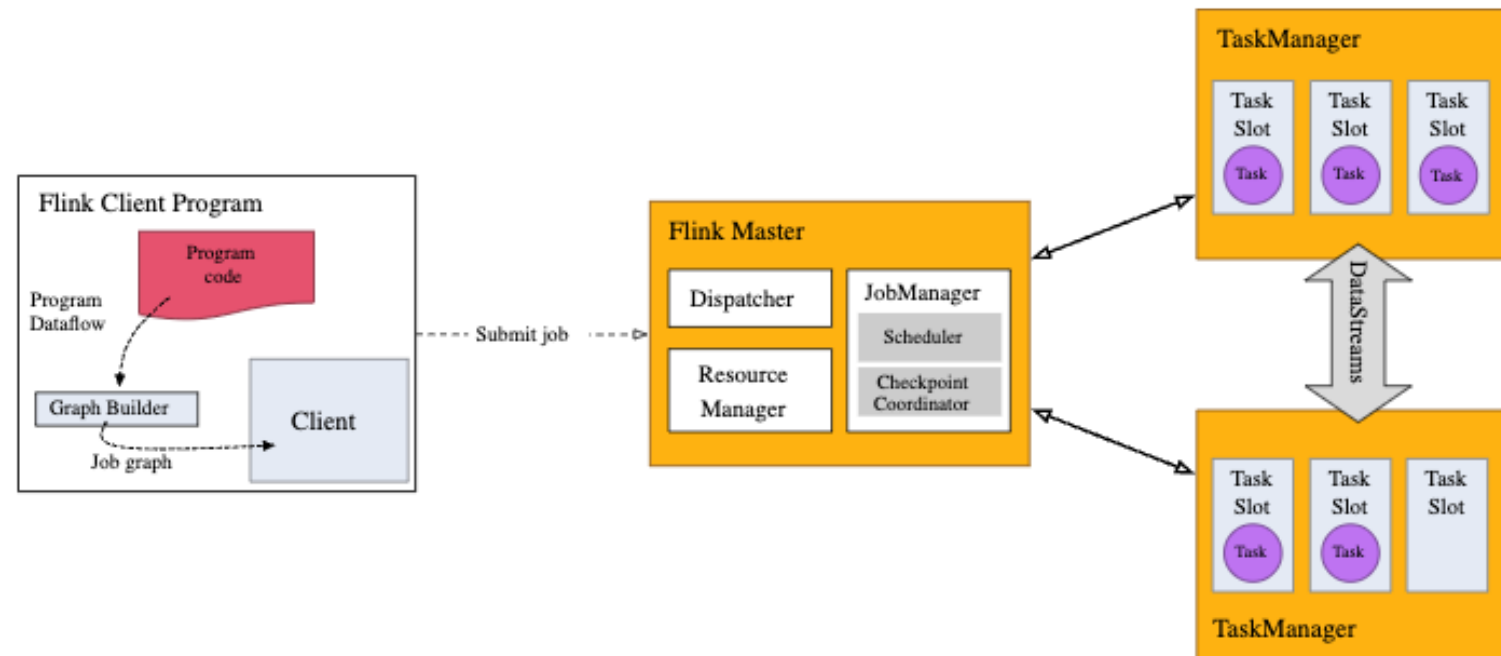


Figure 3-1. Application submission and component interactions



Physical overview Flink UI

Customer Satisfaction Score Prediction Cancel

Job ID	8b5878087e097d555ca2c1dd1cc4a896	Job State	RUNNING 2	Actions	Job Manager Log
Job Type	STREAMING	Start Time	2024-11-28 14:26:41.865	Duration	25s 212ms

[Overview](#) [Exceptions](#) [Data Skew](#) [TimeLine](#) [Checkpoints](#) [Configuration](#)

```
graph LR; S1[Source: Custom Source  
Parallelism: 1] -- "180B" --> S2[Source: Custom Source  
Parallelism: 1]; S1 -- "180B" --> S3[org.myorg.quickstart.operators.connectors.ConnectFeedbackAnd...  
Parallelism: 1]; S2 -- "180B" --> S3;
```

Name	Status	Bytes Received	Records Received	Bytes Sent	Records Sent	Parallelism	Start Time	Duration	End Time	Tasks
Source: Custom Source	RUNNING	0 B	0	0 B	9	1	2024-11-28 14:26:41.846	25s 131ms	~	1
Source: Custom Source	RUNNING	0 B	0	0 B	7	1	2024-11-28 14:26:41.947	25s 130ms	~	1
org.myorg.quickstart.operators.connectors.ConnectFeedbackAnd...	RUNNING	1.03 KB	16	0 B	0	1	2024-11-28 14:26:41.848	25s 129ms	~	1

Operators – graph nodes

- Can do various functions, in various flavours
 - Stateless transformations
 - `map()`, `flatMap()`, `reduce()`, `filter()`, ...
 - Stateful transformations
 - `rich*()` – has context, timer services, etc.
 - Processors
 - Very similar to `map()`, a bit different
 - Connectors
 - `coProcess()`, `richCoProcess()`, `richCoFlatMap()` ...
 - Broadcast functions, union operators, key-by operations

Operators – graph nodes

- Each operator/function is defined as:
 - An interface that you can implement
 - An abstract class that you can extend
- Either way, we can implement our custom logic
- Need to know functional programming!
- Data structure knowledge is a must!

Example of a
stateless map

```
public class MyBeautifulMap implements MapFunction<Foo, Bar>

    @Override
    public Bar map(Foo foo) throws Exception {
        // there was never a better map
        return null;
    }
}
```

Example of a stateful filter

```
public class MyBeautifulRichFilter extends RichFilterFunction<Foo> {  
  
    @Override  
    public void open(Configuration parameters) throws Exception {  
        super.open(parameters);  
        // my perfect implementation  
    }  
  
    @Override  
    public boolean filter(@NotNull Foo foo) throws Exception {  
        // my flawless code  
        return true;  
    }  
  
}
```

Example of a keyed co-process function

```
public class MyWonderfulKeyedCoProcessFunction extends KeyedCoProcessFunction<String, Foo, Bar, HolyGrail> { no usages
    @Override
    public void open(Configuration config) throws Exception {
        super.open(config);
    }

    @Override no usages
    public void processElement1(Foo foo, KeyedCoProcessFunction<String, Foo, Bar, HolyGrail>.Context context, Collector<HolyGrail> collector) throws Exception {
        // context has access to time services (current time, timer setting)
        // collector collects (duh) the output objects, we do not return them - difference between map and process
    }

    @Override no usages
    public void processElement2(Bar bar, KeyedCoProcessFunction<String, Foo, Bar, HolyGrail>.Context context, Collector<HolyGrail> collector) throws Exception {
        // same here, difference is in the first argument
    }
}
```


Machine learning in Flink

Porting Python-trained models, feature
engineering, and model inference logic to Java via
using JPMML libraries

Model representation

- We use **PMML: Predictive Model Markup Language** (like HTML – HyperText Markup Language) – a standard that describes machine learning models via XML (eXtensible Markup Language)
- Idea is to have language-agnostic model representation
 - Do you know how an XGBoostRegressor "looks like"?
 - What about a recurrent neural network? An SVM?
- References:
 - <https://dmg.org/pmml/v4-4-1/GeneralStructure.html>
 - <https://github.com/jpmml/sklearn2pmml>
 - <https://github.com/jpmml/jpmml-evaluator>

The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. A large red speech bubble is centered on the page, containing the text.

Nothing is infinite

State persistence, fault tolerance

State must be
persisted

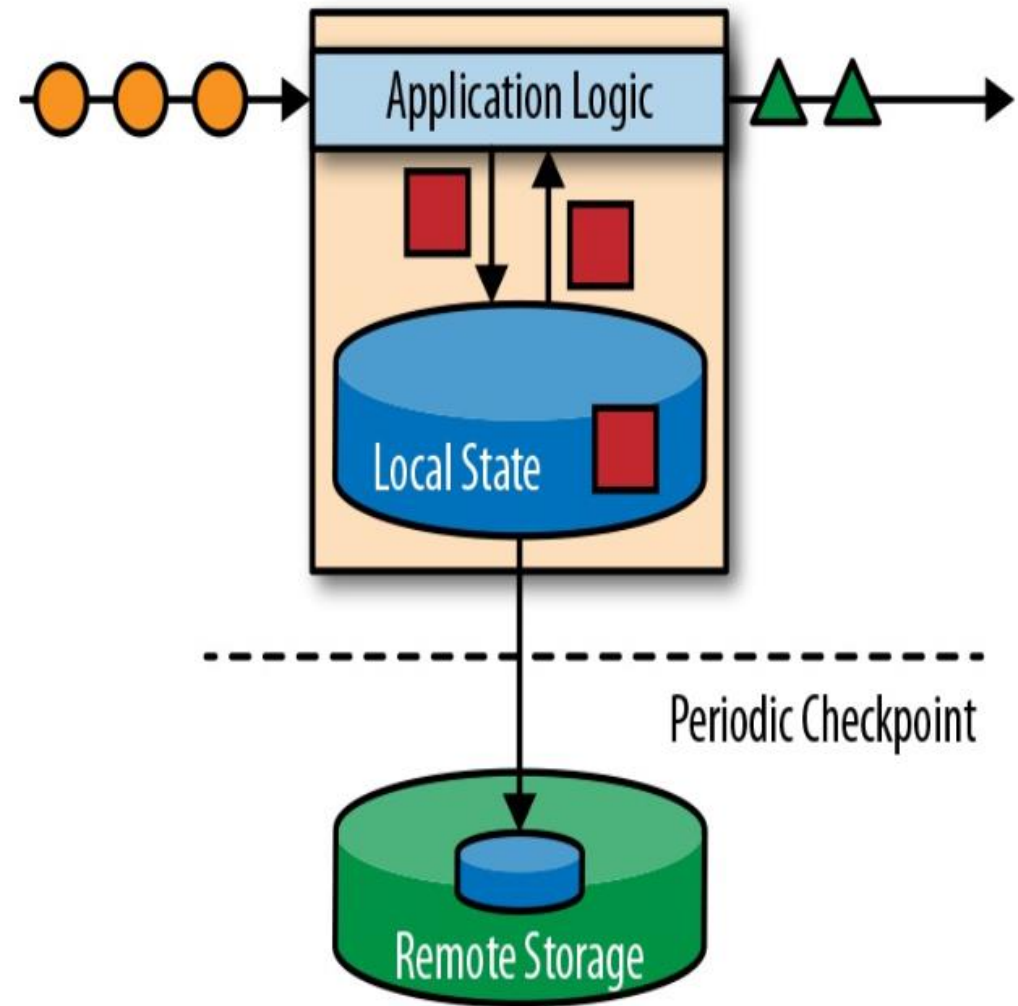
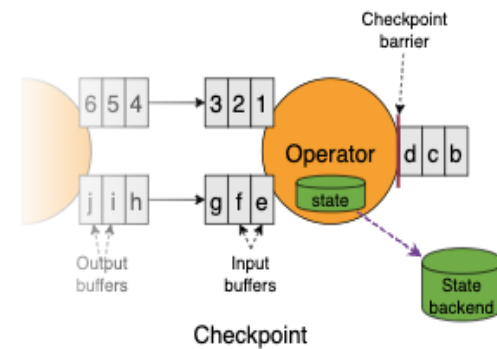
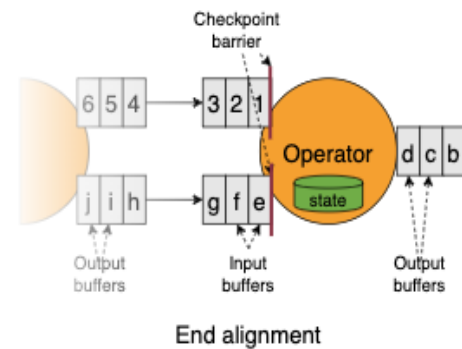
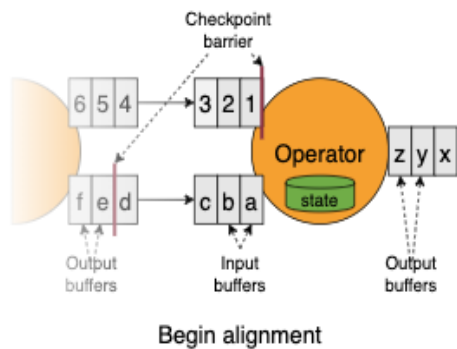
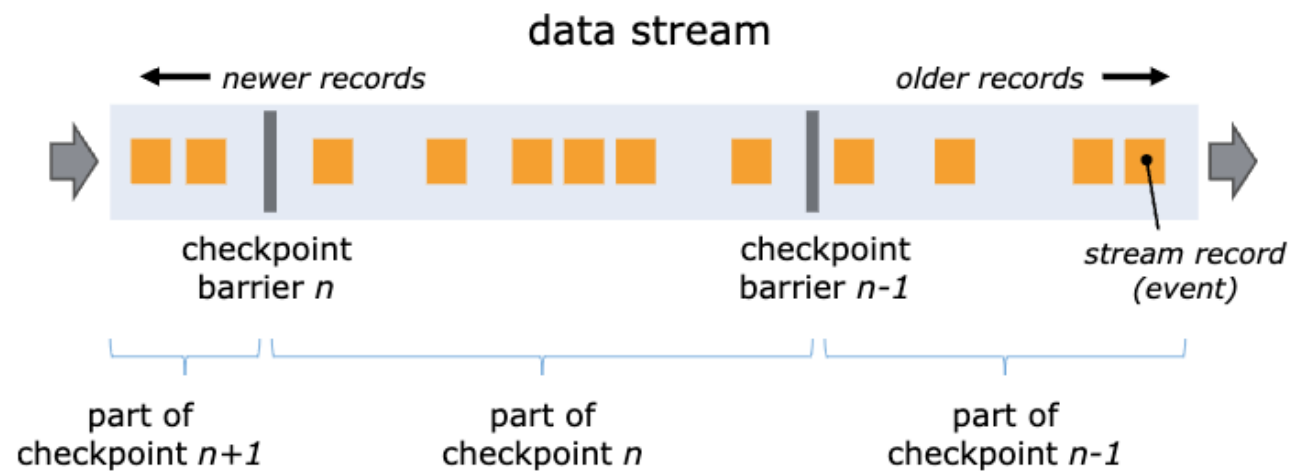


Figure 1-4. A stateful streaming application

Checkpointing mechanism





Docker containers & images

- **Container** is a lightweight, isolated environment that packages code and its dependencies so software runs consistently across systems
- **Image** is a template that defines what's inside of a container, including application files, libraries, and configuration.
- **Docker** is a platform that automates the process of building, running, and managing applications inside containers

Hands-on exercises

There are only 2

Exercise #1

- Flink applications can fail and need to be restarted to read from the last point in time/in the data stream. Sometimes there are infrastructure issues that can cause the application to fail – there's no application exception. Configure your ***execution environment*** to be able to handle at most 2 interruptions/exceptions/failures in any 1-hour period.
- That is, if there are 3 unhandled exceptions in any < 1 hour period, the Flink program should terminate.
- If there's a single unhandled exception, the job should be able to “***restart***” and continue running as if though nothing happened.
- This sounds like a good ***strategy***.

Exercise #2

- Assume the shop opened a d.o.o. in Slovenia and is now operating in the country. The model has not been trained on Slovenian data, and we want to filter it out. Write a function that precedes any data processing and filters out any Slovenian purchases/customers. Add the option to generate Slovenian data objects in the source functions.